

Contents

System Management Controller (SMC) — Architecture & SystemC/TLM-2.0 Modeling Scope	1
1. Purpose & Role of the SMC	2
2. Top-level Architectural View	2
2.1 Key parameters	4
2.2 Memory Map (high level)	5
3. Clock & Reset Domains (modeling-relevant)	6
4. Fabric Behavior (LT abstraction)	6
5. List of IPs to Model in SystemC/TLM-2.0 (LT)	7
5.1 IP Descriptions — What each IP is and its role in the SMC	9
5.2 Modeling priority (recommended)	13
5.3 Firmware availability per IP	13
6. Common Modeling Conventions	17
6.1 TLM-2.0 generic payload extensions	17
6.2 Bus protocols	18
6.3 Loosely-timed timing model	18
6.4 Configuration & build	18
6.5 Verification hooks	18
7. SMC Integration in OCAH SMU	19

System Management Controller (SMC) — Architecture & SystemC/TLM-2.0 Modeling Scope

Source: Open Chiplet Atlas Harness (OCAH) Hardware Specification — Chapter 6, *System Management Controller (SMC)*.

Target modeling style: Accellera SystemC 2.3.x + TLM-2.0 **Loosely-Timed (LT)** with b_transport and DMI where applicable.

1. Purpose & Role of the SMC

The System Management Controller (SMC) is a per-chiplet subsystem that provides:

- Clock, voltage and reset management (PLLs, reset trees, FLR, AVS).
- Hardware platform configuration and bring-up (boot ROM, eFuse, straps).
- System monitoring & telemetry (PVT, ATB telemetry, log engine).
- Inter-processor / inter-chiplet communication (mailboxes, OCTS time sync, I2C/I3C).
- Security-aware fabric (address remap, inbound/outbound filters, protection bits).
- Centralized interrupt management (PLIC, CLINT, per-core watchdogs, BEUs).

In a multi-chiplet SiP each chiplet contains one SMC; the SMC of the **primary** chiplet additionally orchestrates SiP-level tasks for **secondary** chiplets via the OCCP protocol.

The SMC is built around a small RISC-V (Rocket RV64GC) firmware-driven microcontroller cluster (1–4 cores). All control logic lives in firmware; HW IPs are added only when firmware cannot meet performance / determinism / security requirements.

2. Top-level Architectural View

The diagram below is a re-drawn, modeling-oriented view of *Figure 2 — SMC Top-Level Block Diagram* in the OCAH HW spec (Chapter 6). It groups blocks by the SystemC modeling unit defined in §5 of this document and keeps the dual-network fabric, security filters, address remap, CPU cluster and peripheral set explicit.

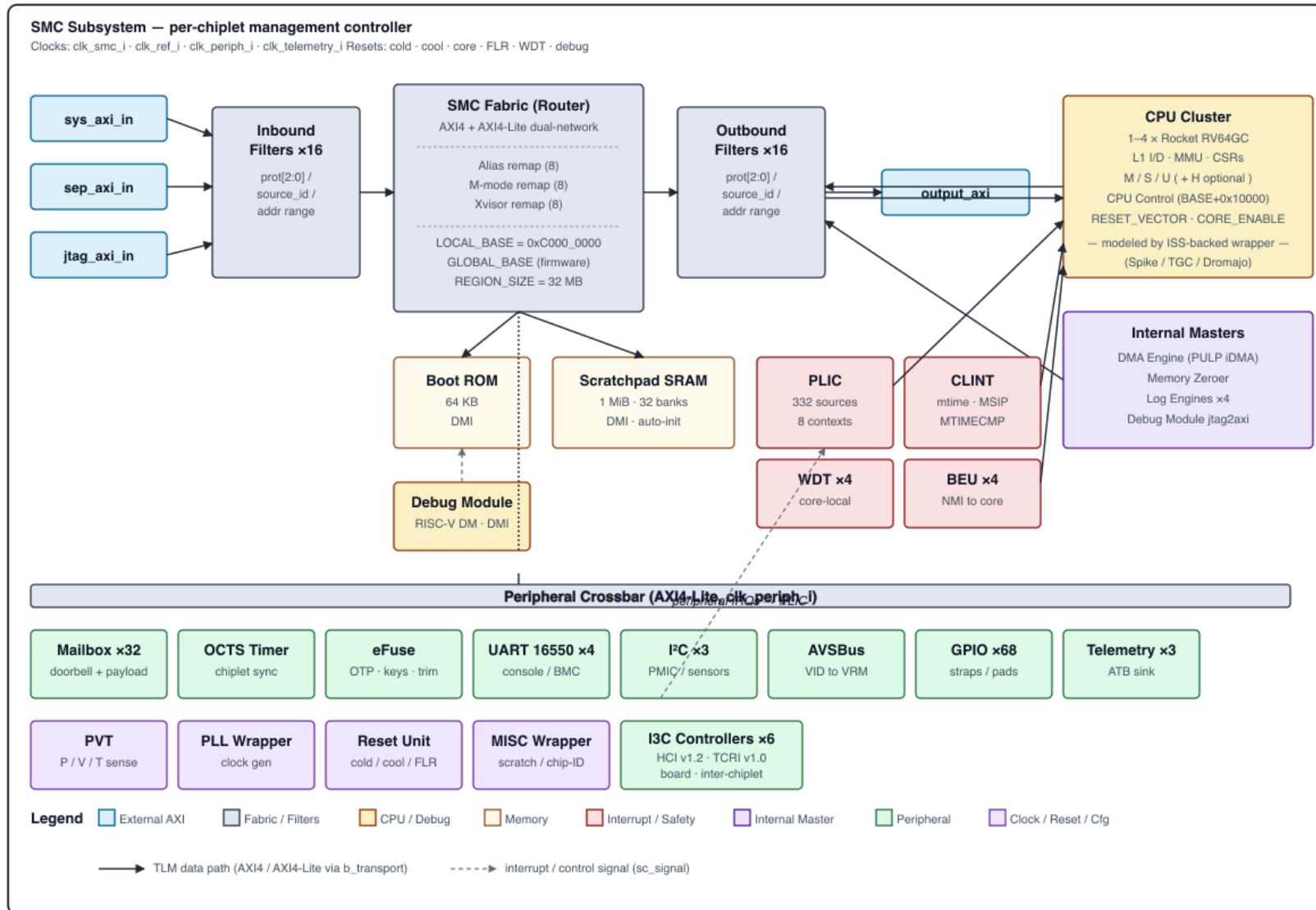


Figure 1: Figure 2 (re-drawn). SMC top-level block diagram, modeling-oriented view. Solid arrows are TLM data paths (b_transport); dashed arrows are interrupt or control signals modelled as sc_signal<bool>. Each block corresponds to one SystemC modeling unit listed in §5.

2.1 Key parameters

Parameter	Value
CPU cores	1–4 Rocket RV64GC
Aperture size	32 MB at SMC-local base 0xC000_0000
AXI4 data width	64-bit
AXI4 system address width	56-bit
AXI4 internal address width	32-bit
Maximum interrupt sources at PLIC	332 (326 active)
Peripheral clock domain	≥ 100 MHz

2.2 Memory Map (high level)

Range (BASE+)	Size	Block
0x000_0000–0x000_0FFF	4 KB	Watchdog Timers (4 cores)
0x000_1000–0x000_1FFF	4 KB	RISC-V Debug Module
0x000_2000–0x000_3FFF	8 KB	Reset Unit, MISC, PLL Wrapper
0x000_4000–0x000_4FFF	4 KB	GPIO Interface (68) + GPIO Control
0x000_7000–0x000_AFFF	16 KB	PVT, AVSBus, I2C, UART
0x000_B000–0x000_FFFF	20 KB	eFuse, Telemetry, OCTS, DTP/DFT
0x001_0000–0x001_7FFF	32 KB	CPU Control, Alias/M-mode/Xvisor remap, Filters
0x001_8000–0x003_7FFF	128 KB	Mailbox (32 channels)
0x003_8000–0x003_83FF	1 KB	DMA + Zeroer
0x004_0000–0x007_FFFF	256 KB	ROM + Scratchpad SRAM
0x040_0000–0x040_1DFF	≈7.5 KB used (4 MB reserved)	I3C Controllers x6 (0x500 stride per instance)
0x100_0000–0x17F_FFFF	8 MB	M-mode remapped region
0x180_0000–0x1FF_FFFF	8 MB	Xvisor remapped region
0x400_0000–0x43F_FFFF	4 MB	PLIC
0x800_0000–0x800_FFFF	64 KB	CLINT
0x801_0000+N×0x1000	4×4 KB	Bus Error Units

LOCAL_BASE = 0xC000_0000 (read-only); GLOBAL_BASE is firmware-programmable (default 0x4000_0000); both alias the same 32 MB aperture.

3. Clock & Reset Domains (modeling-relevant)

Domain	Drives
clk_smc_i	CPU cluster, AXI4 fabric, alias remap, filters
clk_ref_i	CLINT, OCTS, debug, reset CDC anchor, PLL ref
clk_periph_i	AVSBus, I2C, UART, I3C, peripheral crossbar
clk_telemetry_i	Telemetry receiver (sourced by ATB)

Reset hierarchy:

- **Cold Reset** (POR / external) → primary reset.
- **Cool Reset** (BMC / FLR sequence) → primary reset, with isolation.
- **Core Reset** → CPU + private caches, PLIC, CLINT, WDT, BEU.
- **Function Level Reset (FLR)** → PCIe-driven; programmable pre-reset delay (ISOLATE_REQ_FLR_COUNTER_VALUE) and reset hold (ISOLATE_REQ_FLR_RESET_COUNTER_VALUE); auto-bypasses memory repair via `skip_mem_repair_o`.
- **Watchdog Reset / Debug Reset** → core-local.

For SystemC/LT modeling: - Clocks are *abstract* (no waveform). Each module exposes a single `sc_event` based reset and uses `sc_time` deltas for delay annotations. - Reset signals modeled as boolean inputs that trigger `SC_METHOD` initializers.

4. Fabric Behavior (LT abstraction)

The SMC fabric is a dual-network interconnect:

- **AXI4 high-performance network** for CPU cluster, SRAM, DMA, filtering, external AXI ports.
- **AXI4-Lite low-performance network** for peripherals and configuration registers.

Loosely-timed model representation: - A **TLM-2.0 router** module provides a single `tlm_utils::simple_target_socket` per IP and a `tlm_utils::simple_initiator_socket` per master (CPU, DMA, JTAG2AXI, Log Engine, `sys_axi_in`, `sep_axi_in`). - Each transaction carries `tlm_generic_payload` extended with the following sideband fields (see §6.1): - `prot[2:0]` — AXI protection bits. - `source_id`

— SMC_ID / OTHER_ID / MMODE_ID. - cacheable — set by alias remap. - Address remapping (Alias, M-mode, Xvisor) and Inbound/ Outbound filtering are **purely functional** in LT; latency is annotated only when necessary for performance studies (default 0 ns).

5. List of IPs to Model in SystemC/TLM-2.0 (LT)

The table below enumerates the **unit-of-modeling** for the SMC. Each entry corresponds to one SystemC module that will be detailed in 02_SMC_IP_LowLevel_Design.md.

#	IP Module	TLM Role	Bus Protocol Mod- eled	Approx. Space	Reg	Source Section
1	smc_cpu_cluster (Rocket cores wrapper)	Initiator + Target	AXI4 / AXI4-Lite	8 KB CPU Ctrl		\$6.4
2	plic (RISC-V PLIC, 332 sources)	Target	AXI4-Lite	4 MB		\$6.6.2
3	clint (RISC-V CLINT, 4 cores)	Target	AXI4-Lite	64 KB		\$6.6.6
4	wdt (per-core watchdog timer x4)	Target	APB4	1 KB each		\$6.4.7
5	bus_error_unit (per-core BEU x4)	Target	AXI4-Lite	4 KB each		\$6.4.8
6	boot_rom	Target	AXI4-Lite	64 KB (up to)		\$6.5
7	scratchpad_sram (32 banks, 1 MiB)	Target	AXI4	128 KB x2 win- dows		\$6.4.2
8	smc_fabric (router + alias/ M-mode/Xvisor remap)	Router	AXI4 + AXI4-Lite	—		\$6.7
9	axi_filter (in- bound x16 + out- bound x16)	Filter	AXI4 / AXI4-Lite	32 B per filter		\$6.7.8/9
10	dma_engine (PULP iDMA fron- tend+backend+re- sampler)	Initiator + Target	AXI4 + AXI4-Lite	512 B		\$6.8
11	ismp (RISC-V iDMA backend+resampler; NB: iDMA is not enabled in QEMU) (ip cfg)	Target for RISC-V func (bus2mem) del	AXI4-Lite	0.5 B iDMA + 2 B instance (7.5 KB total)		\$6.8.5.1 (iph) / \$6.8.5.2 (iph) / \$6.8.5.3 (iph) / hw/periph/ i3ccore_wrap

The I3C controllers are now available in the OCAH HW tree (`hw/periph/i3ccore_wrap`, parameter `NUM_I3C = 6`) and are included in the modeling scope. Each instance is a MIPI **I3C Basic v1.0/v1.1.1**, **HCI v1.2** and **TCRI v1.0** compliant controller (CHIPS Alliance i3c-core + OCA enhancements) with AXI4-Lite slave register interface and an interrupt line into the PLIC. They replace, rather than reuse, the I²C controller pattern: queue model, CCC engine, IBI handling and DAT/DCT tables make I3C considerably more involved than I²C.

5.1 IP Descriptions — What each IP is and its role in the SMC

The following descriptions summarise *what each IP does* and *why the SMC needs it*. They complement the table above (which gives the modeling parameters) and are intended to be self-contained reading.

CPU & core-private blocks

1. **smc_cpu_cluster** — A wrapper around 1–4 SiFive **Rocket RV64GC** cores that act as the SMC’s firmware engine. Every management decision (boot, FLR, telemetry, OCCP, AVS, security) is taken by software running on these cores; all other IPs exist only to give that firmware deterministic, low-latency hooks into the hardware. The wrapper also holds the per-core L1 caches and the CPU Control register block (alias remap controls, SMP boot release, performance counters).
2. **pllc** — RISC-V **Platform-Level Interrupt Controller** that aggregates up to **332 interrupt sources** (mailboxes, peripherals, telemetry, BEUs, watchdogs, external SiP/SEP signals) and prioritises them for the Rocket cores. Without the PLIC the firmware would have to poll every peripheral; with it, the SMC can react to mailbox traffic, FLR requests, errors and timer events as M-mode external interrupts.
3. **clint** — **Core-Local Interruptor** providing per-hart `mtime`, `mtimecmp`, software (`msip`) and timer (`mtip`) interrupts. It is the canonical RISC-V tick source for the SMC firmware scheduler and is also used by the Linux/Xvisor port that some SiP variants run inside the SMC.
4. **wdt** — A per-core **Watchdog Timer** (×4) that the SMC firmware must kick periodically. If a core hangs it triggers a core-local reset, which is the SMC’s last-resort recovery mechanism for stuck management code; the watchdog interrupt also feeds the PLIC for soft-fail telemetry.
5. **bus_error_unit** — Per-core **Bus Error Unit** (×4) that captures and reports access errors raised by the local AXI fabric (decode error, slave error, filter denial, ECC). It is used both by firmware exception handlers and by the security model: any disallowed access from an external master surfaces here as an interrupt with a captured fault address.

Memories

6. **boot_rom** — Read-only **Boot ROM** holding the immutable first-stage boot loader and the public ROT (root-of-trust) signature roots. After cold reset the Rocket cores fetch from this ROM; it is also the only SMC region addressable before the alias/M-mode remap is programmed.
7. **scratchpad_sram** — On-die **Scratchpad SRAM** (32 banks $\approx$ 1 MiB) that hosts firmware code, stacks, mailbox payloads, log ring buffers and DMA scratch. Because it sits inside the SMC's local AXI fabric it gives the firmware deterministic latency that external DDR cannot guarantee, and it is reachable from the DMA engine and the memory zeroer for fast bulk operations.

Interconnect & security

8. **smc_fabric** — The **dual-network interconnect** (AXI4 + AXI4-Lite) plus the **alias / M-mode / Xvisor address remap** logic. It is what stitches every other IP together: it routes external `sys_axi_in` / `sep_axi_in` / `jtag_axi_in`, the CPU cluster, DMA, log engine and memory zeroer to the local SRAM/ROM, peripheral crossbar and out to `output_axi`. The remap shims let firmware present the same physical memory under different windows for hypervisor and bare-metal contexts.
9. **axi_filter** — Programmable **AXI inbound ($\times 16$) and outbound ($\times 16$) filters** that gate transactions by address range, AXI `prot[2:0]` and `source_id`. They are the SMC's first line of security enforcement: external chiplets and the SEP can only touch SMC resources that firmware has explicitly opened, and SMC-originated traffic to the rest of the SiP is similarly constrained.

Bulk movement

10. **dma_engine** — A **PULP iDMA**-style DMA controller (frontend, backend, request manager) used to move boot images from external memory to scratchpad, to copy mailbox payloads, and to stream telemetry/log data out without burning CPU cycles. It is the only IP besides the CPU that can act as an AXI initiator on the local high-performance network.
11. **memory_zeroer** — A simple **bulk-zeroing engine** that wipes large memory regions on FLR, secure-handover and post-test scrubbing flows. Doing this in hardware makes the FLR latency budget achievable and avoids cache pollution on the Rocket cores.

Inter-processor / inter-chiplet communication

- 12. **mailbox_unit** — Up to **32 doorbell + payload mailbox pairs** used for SMC↔SEP, SMC↔BMC and SMC↔neighbouring-chiplet communication. Each pair has its own interrupt line into the PLIC, which is how OCCP and host-management traffic is signalled.
- 13. **system_timer_octs** — **Open Chiplet Time-Synchronisation** (OCTS) timer that distributes a chiplet-wide synchronous time base via dedicated pulses. Firmware reads it to time-stamp telemetry and to coordinate phase-aligned events (reset deassertion, AVS step, debug capture) across multiple chiplets in a SiP.
- 14. **efuse** — **One-time-programmable fuse controller** with shadow registers, bank model and SHIM. Holds chip ID, security keys, calibration trims and feature-disable bits. Read at boot to populate hardware straps; writeable via a guarded sequence for production programming.

Serial peripherals (for BMC / debug / external sensors)

- 15. **uart_16550** — Four **16550-compatible UARTs** used for firmware console, debug log streaming and out-of-band BMC channels. Standard 16550 register layout makes existing OS / BMC drivers work unchanged.
- 16. **log_engine** — Four **hardware log engines** that DMA structured log records (with OCTS timestamps) from firmware ring buffers into a host-visible region. They offload logging from the CPU and provide a tamper-evident, ordered trace for post-mortem debug.
- 17. **i2c_controller** — Three **OpenTitan-style I²C controllers** for talking to off-die devices: PMIC/VRM, board sensors, EEPROMs and other BMC-side components. They can act both as host (firmware-driven probes) and target (BMC-initiated management traffic).
- 17a. **i3c_controller_wrap** — Six **MIPI I³C** controllers (`i3ccore_wrap`, parameter `NUM_I3C = 6`). Per the OCAH integration the canonical role split is: instance **[0]** for board-level communication, instances **[1]–[2]** for inter-chiplet communication, and instances **[3]–[5]** as spare/expansion. Each instance is **HCI v1.2** compliant with full **active-controller**, **secondary-controller** and **target** modes, plus **TCRI v1.0** recovery for secure firmware download. Compared to I²C, the I³C controllers add: descriptor-based **HCI queues** (Command/Response/TX/RX/IBI), **In-Band Interrupts** (with an OCA-specific reduced-latency mechanism that lets a target inject an IBI during the controller's broadcast phase), **Common Command Codes** (SETDASA, SETNEWDA, ENEC/DISEC, GETPID, GETSTATUS, etc.), and **DAT (128 entries)** / **DCT (128 entries)** device tables. They are I²C-backwards-

compatible (Standard/Fast/Fast-mode-Plus) and run up to **12.5 MHz Push-Pull** I3C. From the SMC's point of view they are AXI4-Lite slaves (32-bit accesses only, 0x500 bytes each) on `clk_periph_i` whose **6 interrupt outputs** are CDC-crossed to `clk_smc_i` and routed into PLIC peripheral sources [17:12].

- 18. **avsbus_controller** — **AVSBus master** used to drive the on-board voltage regulators for adaptive voltage scaling. The SMC firmware combines PVT readings and workload telemetry to compute new VID codes and push them out through this block.
- 19. **gpio** — **68 general-purpose I/O pins** plus the central GPIO control block. Used for board straps, debug LEDs, reset/wake signalling, and any per-board glue the SMC needs to expose.
- 20. **telemetry_receiver** — Three **ATB (ARM Trace Bus) sinks** that ingest streaming PVT, performance and debug telemetry from the rest of the SiP and present it to the SMC firmware (and through it to the BMC/host). This is the SMC's main way of "seeing" what the compute chiplets are doing.

Clock, reset & infrastructure

- 21. **reset_unit** — Centralised **reset controller**. Sequences cold/cool/core/FLR/watchdog/debug resets, hosts the FLR pre-reset and reset-hold counters, and exposes the `skip_mem_repair` strap. Every other SMC IP and every external function that the SMC governs receives its reset from here.
- 22. **pll_wrapper** — **PLL configuration wrapper** that exposes PLL control/status registers (`refdiv`, `fbdiv`, `postdiv`, `lock`, `bypass`) for `clk_smc`, `clk_periph` and friends. Firmware programs it during boot and during DVFS transitions; it is the source of every SMC clock domain.
- 23. **misc_wrapper** — Catch-all **chip-config / scratch-register block**. Holds NDM (non-debug-module) reset strap, chip-id mirror, scratch registers used by firmware for inter-stage handoff, and miscellaneous integration glue that does not justify its own IP.
- 24. **debug_module** — RISC-V **Debug Module** (with built-in `jtag2axi` initiator). Provides JTAG access into the SMC: halt/resume/single-step the Rocket cores, peek/poke any AXI-reachable register, and load firmware over JTAG. Required for silicon bring-up and post-silicon debug.

25. **pvt_wrapper — Process / Voltage / Temperature sensor wrapper**. Surfaces on-die thermal and voltage readings used by AVS, throttling and telemetry. The SMC firmware polls (or is interrupted by threshold crossings from) this block to keep the chiplet within its operating envelope.

5.2 Modeling priority (recommended)

Priority	Modules	Rationale
P0 (must have)	smc_fabric, axi_filter, smc_cpu_cluster (stub), plic, clint, boot_rom, scratchpad_sram, reset_unit, misc_wrapper, mailbox_unit, system_timer_octs, uart_16550, gpio, efuse	Required to boot SMC firmware up to OCCP loop.
P1 (must have soon)	dma_engine, memory_zeroer, log_engine, i2c_controller, avsbus_controller, pvt_wrapper, telemetry_receiver, pll_wrapper, wdt, bus_error_unit, i3c_controller_wrap (instance [0] only — board comms)	Required for full functional system management workloads.
P2	debug_module, i3c_controller_wrap (instances [1]–[5], inter-chiplet + spare)	Inter-chiplet I3C and JTAG/debug flows can be brought up after P0/P1 stabilises.

5.3 Firmware availability per IP

This sub-section maps each modeling unit from §5 to the firmware that already exists in the OCAH repository (under `fw/smc/`). The aim is to make it obvious which IPs the SystemC model can be exercised against with **real, in-tree firmware** versus those that will need stub drivers or hand-written register pokes from the test bench.

The repository organises SMC firmware into:

- `fw/smc/common/` — runtime & C drivers (Freedom Metal-style `metal/*` API plus Tenstorrent additions `tt_*`).
- `fw/smc/common/drivers/` — vendor drivers (SiFive `sifive_*`, RISC-V `riscv_*`).

- fw/smc/common/metal/ — Freedom Metal headers and platform glue.
- fw/smc/prod_rom/ — production boot ROM (boot loader, OCCP server, ROM-resident drivers).
- fw/smc/tests/ — bare-metal feature & sanity tests run on the SMC core.
- fw/smc/tests_rom/ — boot ROM resident tests (OCCP, OCTS, secure-boot, dual-I²C, I3C, etc.).
- fw/smc/rust/app/ — Rust firmware target (multi-hart sanity / scratch-register exerciser).

Status legend

-  **Driver + tests** — at least one *.c driver/abstraction *and* one dedicated test in fw/smc/tests/ or fw/smc/tests_rom/.
-  **Tests only** — no standalone driver source file; firmware exercises the IP via raw register pokes inside test code (and/or smc_defines.h / smc_top_regs.h register-map headers).
-  **Driver only / indirect** — driver source exists but no dedicated on-target test; the IP is normally exercised as a side-effect of another test (e.g. CLINT timer used by every timed test).
-  **None** — no firmware artefacts in tree (covered, if at all, from the external test bench / DV).

All paths are relative to fw/smc/. Inside cells, each driver / runtime file and each test directory is on its own line so that the table renders cleanly in PDF as well as on screen.

#	IP Module	Status	Drivers / runtime	Dedicated tests
1	smc_cpu_cluster	✓	common/crt0.S entry.S vector.S trap.S init.c common/pmp.c privilege.c synchronize_harts.c common/hpm.c drivers/riscv_cpu.c rust/app/src/ main.rs rust/app/src/ smc_test.rs	coremark dhrystone whetstone quicksort mergesort matrix_mult mutex_semaphore_sanity hello_world hello_world_multicore default_reg_rd version_id
2	plic	✓	common/drivers/ riscv_plic0.c (+.h) common/drivers/ sifive_global- external- interrupts0.c (+.h) common/drivers/ sifive_local- external- interrupts0.c (+.h) common/drivers/ sifive_clic0.c (+.h) common/ interrupt.c common/ tt_smc_interrupts.c (+.h) common/tt_smc_lcore_interrupts.h	plic_sanity external_interrupts
3	clint	●	common/drivers/ riscv_clint0.c (+.h) common/time.c common/ timer.c common/rtc.c	(none dedicated; consumed by every timer-driven test)
14a	isr_handlers	✓	common/i2c.c common/time.c common/rtc.c common/timer.c common/riscv_timer.c common/riscv_timer0.c common/riscv_timer1.c common/riscv_timer2.c common/riscv_timer3.c common/riscv_timer4.c common/riscv_timer5.c common/riscv_timer6.c common/riscv_timer7.c common/riscv_timer8.c common/riscv_timer9.c common/riscv_timer10.c common/riscv_timer11.c common/riscv_timer12.c common/riscv_timer13.c common/riscv_timer14.c common/riscv_timer15.c common/riscv_timer16.c common/riscv_timer17.c common/riscv_timer18.c common/riscv_timer19.c common/riscv_timer20.c common/riscv_timer21.c common/riscv_timer22.c common/riscv_timer23.c common/riscv_timer24.c common/riscv_timer25.c common/riscv_timer26.c common/riscv_timer27.c common/riscv_timer28.c common/riscv_timer29.c common/riscv_timer30.c common/riscv_timer31.c	common/i2c.c common/time.c common/rtc.c common/timer.c common/riscv_timer.c common/riscv_timer0.c common/riscv_timer1.c common/riscv_timer2.c common/riscv_timer3.c common/riscv_timer4.c common/riscv_timer5.c common/riscv_timer6.c common/riscv_timer7.c common/riscv_timer8.c common/riscv_timer9.c common/riscv_timer10.c common/riscv_timer11.c common/riscv_timer12.c common/riscv_timer13.c common/riscv_timer14.c common/riscv_timer15.c common/riscv_timer16.c common/riscv_timer17.c common/riscv_timer18.c common/riscv_timer19.c common/riscv_timer20.c common/riscv_timer21.c common/riscv_timer22.c common/riscv_timer23.c common/riscv_timer24.c common/riscv_timer25.c common/riscv_timer26.c common/riscv_timer27.c common/riscv_timer28.c common/riscv_timer29.c common/riscv_timer30.c common/riscv_timer31.c
15	uart	✓	common/uart.c common/uart0.c common/uart1.c common/uart2.c common/uart3.c common/uart4.c common/uart5.c common/uart6.c common/uart7.c common/uart8.c common/uart9.c common/uart10.c common/uart11.c common/uart12.c common/uart13.c common/uart14.c common/uart15.c common/uart16.c common/uart17.c common/uart18.c common/uart19.c common/uart20.c common/uart21.c common/uart22.c common/uart23.c common/uart24.c common/uart25.c common/uart26.c common/uart27.c common/uart28.c common/uart29.c common/uart30.c common/uart31.c	common/uart.c common/uart0.c common/uart1.c common/uart2.c common/uart3.c common/uart4.c common/uart5.c common/uart6.c common/uart7.c common/uart8.c common/uart9.c common/uart10.c common/uart11.c common/uart12.c common/uart13.c common/uart14.c common/uart15.c common/uart16.c common/uart17.c common/uart18.c common/uart19.c common/uart20.c common/uart21.c common/uart22.c common/uart23.c common/uart24.c common/uart25.c common/uart26.c common/uart27.c common/uart28.c common/uart29.c common/uart30.c common/uart31.c

5.3.1 Roll-up summary

Status	Count	IPs
 Driver + tests	13	smc_cpu_cluster, plic, wdt, bus_error_unit, boot_rom, scratchpad_sram, dma_engine, memory_zeroer, efuse, uart_16550, i2c_controller, i3c_controller_wrap, gpio, pll_wrapper (13 entries)
 Driver only / indirect tests	4	clint, smc_fabric, log_engine, misc_wrapper
 Tests only (raw register access)	7	mailbox_unit, system_timer_octs, avsbus_controller, telemetry_receiver, reset_unit, debug_module, pvt_wrapper
 None on target	1	axi_filter

5.3.2 Implications for SystemC bring-up

- **All P0 (boot-to-OCCP) IPs except `axi_filter`** have at least one on-target firmware artefact, so the SystemC top model can be brought up against real firmware as soon as the corresponding modules are written. The CPU runtime, Boot ROM (with OCCP server), PLIC, CLINT, scratchpad SRAM, mailbox, OCTS, UART, GPIO, eFuse and reset-unit are all directly addressable from in-tree code.

- **axi_filter** has no on-target driver because firmware programs it through plain register writes via the SMC fabric register-map headers; the SystemC model therefore needs to expose the same register layout faithfully but can be exercised by a thin test-bench shim rather than by ported firmware.
- **misc_wrapper** and **smc_fabric** are intentionally driven by raw register pokes (chip-id mirror, scratch registers, alias remap) inside `init.c / prod_rom/lib/src/init.c`. The SystemC models must match the register layout in `prod_rom/registers/smc_top_regs.h` exactly — there is no abstraction layer to absorb mismatches.
- **log_engine**: only partial firmware coverage today (`sifive_trace.c`). When the full SMC log-engine model is brought up, a corresponding on-target driver should be added under `fw/smc/common/` so that log-engine ring-buffer flushing can be tied to the OCTS time base via firmware rather than DV stimulus.
- **debug_module**: by design, JTAG2AXI is driven from the DV test bench (External JTAG agent), not from on-target firmware. The SystemC `debug_module` will be exercised primarily by the test bench's JTAG2AXI initiator socket; the only on-target test is `dft_ctrl_sanity` which checks the DM-side scan/test-mode straps.

5.3.3 Source of truth for register addresses

For all IPs marked 🟡 above, the SystemC implementer should treat the following two headers as the **authoritative register-address contract**, since the firmware's tests poke them directly without an intervening driver:

- `fw/smc/prod_rom/registers/smc_top_regs.h` — generated SMC top-level register map.
- `fw/smc/common/smc_defines.h` and `fw/smc/common/quasar_defines.h` — manually maintained address constants used by `init.c`, OCCP, and tests.

A SystemC model whose register offsets match these headers will boot the existing firmware and pass the corresponding tests without code changes on the firmware side.

6. Common Modeling Conventions

6.1 TLM-2.0 generic payload extensions

All SMC IPs share a common GP extension header. Define once in `smc_tlm_extensions.h`:

```
struct smc_axi_extension : tlm::tlm_extension<smc_axi_extension> {
    uint32_t source_id = 0;           // SMC_ID / OTHER_ID / MMODE_ID
```

```

uint8_t prot      = 0;          // AXI prot[2:0]
bool      cacheable = false;
bool      non_secure = false;   // alias of prot[1]
uint16_t axi_id   = 0;
uint8_t axi_user   = 0;
// ... clone(), copy_from() boilerplate ...
};

```

6.2 Bus protocols

- **AXI4 / AXI4-Lite / APB4** are *not* modeled at signal level. They are abstracted to TLM-2.0 generic payload + `smc_axi_extension`.
- **Endianness:** little-endian throughout.
- **Alignment:** 32-bit registers on 4-byte boundaries; 64-bit registers on 8-byte boundaries.

6.3 Loosely-timed timing model

- IPs use `b_transport()` only.
- Annotated delays are coarse: zero by default for register accesses, and approximate values for long operations (e.g. eFuse program $\approx$ tens of μ s, DMA per-burst $\approx$ 1 cycle/beat at 1 GHz default).
- Quantum keepers (`tlm_utils::tlm_quantumkeeper`) wrap CPU cluster, DMA engines, the memory zeroer, the log engines, and all bus-side initiators (I2C, I3C, AVSBus, Mailbox payload path).
- **Temporal decoupling for simulation speed-up** is a first-class concern; project-wide policy, defaults (1 μ s global quantum, S1–S4 sync rule, fast vs lock-step run modes) and per-IP buckets are specified in `02_SMC_IP_LowLevel_Design.md` **§0 — Temporal Decoupling for Simulation Speed-Up**.

6.4 Configuration & build

- One CMake target per IP, plus a top-level `libsmc.a`.
- All registers generated from JSON specifications using `scc::scc_register` or hand-written `register_bank` template.
- Per-module Tracer produces VCD/CSV traces.

6.5 Verification hooks

- Each IP exposes a control socket (`tlm_utils::simple_target_socket<..., 32>`) for back-door read/write from the testbench.
- Every IP has a `dump_state()` method and a published list of register-bit assertions.

7. SMC Integration in OCAH SMU

When wired into the OCAH **SMU** (Sec. 16.1 of the spec), the SMC exposes:

- 4 AXI ports: `sys_axi_in`, `sep_axi_in`, `jtag_axi_in`, `output_axi`.
- 32 MB aperture; firmware programs `GLOBAL_BASE`.
- Clocks: `clk_smc_i`, `clk_ref_i`, `clk_periph_i`, `clk_telemetry_i`.
- Resets: `rst_cold_ni`, `powergood_i`, optional override resets.
- Aggregated `sync_irq_o`; per-mailbox interrupt outputs; SEP mailbox interrupt inputs.

For SystemC TLM-2.0 modeling:

- The **SMC top module** (`smc_top.h`) instantiates all P0/P1 IPs and binds them through the fabric router.
- It exposes 4 TLM target/initiator sockets and a `gpio_pads` array.
- Reset logic is a single `reset_controller` SC_MODULE that issues reset events into each IP via `sc_event`-based subscription.

This document defines **what** is modeled. The companion document `02_SMC_IP_LowLevel_Design.md` defines **how** each IP is modeled in SystemC/TLM-2.0, including class hierarchy, registers, and integration. The companion document `03_SMC_Test_Plan.md` defines the verification plan for the resulting model library.